

Projeto: Testes Unitários de JavaScript com Jest

Autor:

Guilherme de Oliveira Cantarino

Ferramentas:

Node.JS, Jest, navegadores (Chrome, Edge e Firefox)

Execução dos testes:

(Git) <https://github.com/GksOC/EBAC/tree/master/JS>

Testes unitários separados na pasta:

- ./JS/unit-tests

Diretórios fonte utilizados para a realização dos testes:

- ./JS/aulas
- ./JS/Desafio

Análise dos resultados:

Os resultados atingiram métricas satisfatórias, salvo o arquivo “calculadora.js”

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	86.66	87.17	93.75	86.2	
Desafio	100	100	100	100	
usuario.js	100	100	100	100	
aulas	81.81	84.37	91.66	80.95	
acesso-evento.js	100	100	100	100	
calculadora.js	72.41	70.58	88.88	72.41	61-79
pode-votar.js	100	100	100	100	
verificar-idade.js	100	100	100	100	
Test Suites: 5 passed, 5 total					
Tests: 34 passed, 34 total					
Snapshots: 0 total					
Time: 6.018 s					
Ran all test suites.					

No entanto, há um motivo para isso acontecer:

Nas linhas de 61-79 contém uma entrada que não é reconhecida nativamente pelo Node.JS que é o “prompt()” Esta função se comunica diretamente com o navegador para a obtenção (genérica) de um valor de entrada.

```
ligarCalculadora(){
  let index = prompt(`
    Escolha sua operação:
    1 - somar
    2 - subtração
    3 - divisão
    4 - multiplicação
    5 - potenciação
    6 - raiz
  `);

  if(isNaN(index) || (index < 1 && index > 6) ){
    alert("Entrada inválida :(");
  }else{
    let numA = prompt("Digite o primeiro número: ");
    numA = parseInt(numA);
    let numB = prompt("Digite o segundo número: ");
    numB = parseInt(numB);

    document.getElementById('calculadora').innerHTML += `
      O resultado da expressão é: ${operacao(index - 1, numA, numB)} <br>
    `;
  }
}
```

Portanto, para que o teste tenha cobertura completa, o seguinte plano foi traçado:

- Criar uma branch
- Modificar a estrutura para que possa receber entradas nativas do Node.JS
- Realizar os testes de cobertura adequados
- Confirmar que código está pronto para a implantação final

Aqui está o código adaptado:

```
ligarCalculadora(index, numA, numB){
  if(isNaN(index) || (index < 1 && index > 6) ){
    alert("Entrada inválida :(");
  }else{
    document.getElementById('calculadora').innerHTML += `
      O resultado da expressão é: ${this.operacao(index - 1, numA, numB)} <br>
    `;
  }
}
```

Através do particionamento de equivalência, as entradas desta forma terão resultados semelhantes ao do usuário digitar no prompt do navegador.

Apesar de apenas “index” passar por uma validação de entrada, “numA” e “numB” são verificados dentro da função “operacao()”.

Cuidados adicionais foram cuidados para que a referência original do objeto não seja esquecida, a palavra chave “this” foi adicionada à função “operacao()” para

que a chamada seja devidamente realizada, pois a última referência do objeto é a própria função “ligarCalculadora()”, então o “this” garante que estamos referenciando a classe Calculadora.

Percepções sobre a qualidade:

A cobertura está boa, mas agora que estamos cobrindo uma nova função, além da verificação já feita com os números, é necessário lembrar que o “index” precisa ser **inteiro**, portanto obrigatoriamente devemos refatorar a verificação do valor do “index” para **não permitir** valores **float**, seja enviando uma mensagem de erro ou utilizando floor().

Sobre os demais testes de cobertura, foram observados testes de borda, valor limite e até entradas inválidas. Eles também abrangem todas as combinações possíveis de acordo com suas categorias. Até mesmo testes em que apenas 1 valor de 3 parâmetros determina o resultado, ainda sim se vê combinações com os outros 2 valores restantes, uma ótima abordagem para identificar possíveis brechas ou bugs e evitar problemas a longo prazo.

Conclusão:

Os testes já têm resultados satisfatórios para uso geral, mas ainda faltam implementações de cobertura mínima (100% de instruções percorridas). O recomendado é que, além da Cobertura de Instruções, também seja feita a Cobertura de Decisões ou, se possível, Cobertura de Caminhos.

Lembrando que não foram levantados requisitos de DOD (Definition Of Done) para estes testes além de /Desafios/usuario.js, mas que ainda são muito simplificados comparado ao contexto de aplicação real. Contudo, todo o rigor técnico foi aplicado quando possível para garantir a demonstração do caráter de QA.